



מבוא למדעי המחשב מ'ח' (234114 \ 234117)

סמסטר חורף תשפ"א

מבחן מסכם מועד א', 22 בפברואר 2021

2	3	4	1	1	
---	---	---	---	---	--

רשום/ה לקורס:

--	--	--	--	--	--	--	--	--	--

מספר סטודנט:

משך המבחן: 3 שעות.

חומר עזר: אין להשתמש בכל חומר עזר.

הנחיות כלליות:

- מלאו את הפרטים בראש דף זה ובדף השער המצורף, בעט בלבד.
- בדקו שיש 22 עמודים (4 שאלות) במבחן, כולל עמוד זה.
- כתבו את התשובות על טופס המבחן בלבד, במקומות המיועדים לכך. שימו לב שהמקום המיועד לתשובה אינו מעיד בהכרח על אורך התשובה הנכונה.
- העמודים הזוגיים בבחינה ריקים. ניתן להשתמש בהם כדפי טיוטה וכן לכתיבת תשובותיכם. סמנו טיוטות באופן ברור על מנת שהן לא תיבדקנה.
- יש לכתוב באופן ברור, נקי ומסודר, ובעט בלבד.
- בכל השאלות, הנכם רשאים להגדיר ולממש פונקציות עזר כרצונכם. לנוחיותכם, אין חשיבות לסדר מימוש הפונקציות בשאלה, ובפרט ניתן לממש פונקציה לאחר השימוש בה.
- אלא אם כן נאמר אחרת בשאלות, אין להשתמש בפונקציות ספריה או בפונקציות שמומשו בכיתה, למעט פונקציות קלט/פלט והקצאת זיכרון (`malloc`, `free`). ניתן להשתמש בטיפוס `bool` המוגדר ב-`stdbool.h`.
- אין להשתמש במשתנים סטטיים וגלובאליים אלא אם נדרשתם לכך מפורשות.
- כשאתם נדרשים לכתוב קוד באילוצי סיבוכיות זמן/מקום נתונים, אם לא תעמדו באילוצים אלה תוכלו לקבל בחזרה מקצת הנקודות אם תחשבו נכון ותציינו את הסיבוכיות שהצלחתם להשיג.
- נוהל "לא יודע": אם תכתבו בצורה ברורה "לא יודע/ת" על שאלה (או סעיף) שבה אתם נדרשים לקודד, תקבלו 20% מהניקוד. דבר זה מומלץ אם אתם יודעים שאתם לא יודעים את התשובה.
- נוסחאות שימושיות:

$$1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} = \Theta(\log n) \quad \log 1 + \log 2 + \dots + \log n = \Theta(n \log n)$$

$$1^k + 2^k + 3^k + \dots + n^k = \Theta(n^{k+1}) \quad 1 + k + k^2 + k^3 + \dots + k^n = \Theta(k^n), \quad k > 1$$

צוות הקורס 234114/7

מרצים: פרופ' תומר שלומי (מרצה אחראי), פרופ' ניר אילון, מר איהאב וותד, גב' יעל ארז **מתרגלים:** גיא בר שלום, מג'ד חורי, עמית ברכה, קטרין חדאד, רואי בנימין, תומר לנגה, מתן ממיסטבלוב, יבגני גוגין, ירון ח'י, תום הרמן, עדי גרוס, הראל וקנין, סמיר מסעד, דמיטרי רבינוביץ' (מתרגל אחראי).

בהצלחה!

[illegible]



שאלה 1 (25 נקודות):

א. (8 נקודות) חשבו את סיבוכיות הזמן והמקום של הפונקציה $f1$ המוגדרת בקטע הקוד הבא, כפונקציה של n . אין צורך לפרט שיקולים. חובה לפשט את הביטוי ככל שניתן.

```
void aux(int n) {
    while (n > 0) {
        printf("%d", n);
        n--;
    }
}

long f1(int n)
{
    long sum = 0;
    for (int i = 1; i * i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            aux(i * j);
            sum += i * j;
        }
    }
    return sum;
}
```

סיבוכיות זמן: $\Theta(\text{_____})$ סיבוכיות מקום: $\Theta(\text{_____})$

ב. (9 נקודות) חשבו את סיבוכיות הזמן והמקום של הפונקציה $f2$ המוגדרת בקטע הקוד הבא, כפונקציה של n . אין צורך לפרט שיקולים. חובה לפשט את הביטוי ככל שניתן.

```
void f2(int n)
{
    int k = n, p = 1;
    while (--k && k % 3 != 0)
        p *= n;

    free(malloc(p * p));
}
```

סיבוכיות זמן: $\Theta(\text{_____})$ סיבוכיות מקום: $\Theta(\text{_____})$

[illegible]



ג. (8 נקודות): חשבו את סיבוכיות הזמן והמקום של הפונקציה f3 המוגדרת בקטע הקוד הבא, כפונקציה של n. אין צורך לפרט שיקולים. חובה לפשט את הביטוי ככל שניתן.

```
void aux(int n)
{
    if (n <= 1)
        return;

    for (int i = 0; i < n; ++i)
        printf("#");

    aux(n / 3);
    aux(n / 3);
}

void f3(int n)
{
    int s = 0;
    for (int i = 0; i < n; ++i)
        s += i;

    aux(s);
}
```

סיבוכיות מקום: $\Theta(\quad)$

סיבוכיות זמן: $\Theta(\quad)$

[illegible]



שאלה 2 (25 נק')

ממשו את הפונקציה המקבלת מערך של מספרים שלמים a ואת הגודל שלו n שחתימתה:

```
int index_of_equal_sum_left_right(int a[], int n);
```

הפונקציה מחזירה אינדקס לתא במערך, כך שסכום האיברים מימינו שווה לסכום האיברים משמאלו (לא כולל האיבר). אם לא קיים אינדקס כזה, הפונקציה תחזיר -1.

לדוגמה: אם נתון המערך

-3	1	4	-1	5	9	-2	-6
----	---	---	----	---	---	----	----

אז הפונקציה תחזיר את האינדקס 4, מאחר וסכום האיברים משמאל לאיבר באותו מיקום (5) שווה לסכום האיברים מימינו ושווה ל-1.

הערות:

- אין חשיבות לאיזה מבין האינדקסים האפשריים יוחזר, כל עוד האיבר באינדקס זה מקיים את הדרישות.
- אם אין אינדקס במערך המקיים את התנאי לעיל הפונקציה תחזיר -1.

דרישות:

- ממשו את הפונקציה בסיבוכיות זמן $O(n)$ וסיבוכיות מקום $O(1)$.
- אם לא עמדתם בדרישות הסיבוכיות, אנא ציינו כאן את הסיבוכיות שהגעתם אליה:

זמן _____ מקום נוסף _____

int index_of_equal_sum_left_right (int a[], int n)
{

[illegible]

[illegible]

[illegible]



שאלה 3 (25 נקודות) :

מחרוזת פלינדרום (palindrome) היא מחרוזת שאם קוראים אותה מהתחלה לסוף או מהסוף להתחלה מקבלים את אותו רצף האותיות. למשל "radar" היא פלינדרום ואילו "software" איננה פלינדרום.

עליכם לממש את הפונקציה שחתימתה:

```
bool rearrangeStringToPalindrome(char* str);
```

הפונקציה מקבלת מחרוזת המורכבת מאותיות קטנות באנגלית ומחזירה true במידה ואפשר לשנות את סדר האותיות במחרוזת הקלט כדי ליצור פלינדרום, ו- false אחרת.

במידה ואפשר לשנות את סדר האותיות באופן שמייצר פלינדרום (כלומר שהפונקציה מחזירה true), עליכם לשנות את תוכן המחרוזת שיכיל פלינדרום חוקי כלשהו שאפשר לקבל משינוי סדר האותיות.

דוגמאות:

```
char str[] = "hello";
rearrangeStringToPalindrome(str); // returns 'false'
printf("%s", str); // Output: "hello" (no change)

char str[] = "adradr";
rearrangeStringToPalindrome(str); // returns 'true'
printf("%s", str); // Possible Outputs: "ardra", "radar"

char str[] = "ayay";
rearrangeStringToPalindrome(str); // returns 'true'
printf("%s", str); // Possible Outputs: "yaay", "ayya"
```

הערות:

- ניתן להניח כי המילים מורכבות מאותיות אנגליות קטנות בלבד.
- מחרוזת ריקה הינה פלינדרום.

דרישות:

- סיבוכיות זמן $O(n)$, כאשר n הינו אורך של המחרוזת str , סיבוכיות מקום $O(1)$.

אם לפי חישוביכם לא עמדתם בדרישות הסיבוכיות אנא ציינו כאן את הסיבוכיות שהגעתם אליה:

זמן _____ מקום נוסף _____



```
bool rearrangeStringToPalindrome(char* str)
```

```
{
```

[illegible]

[illegible]



שאלה 4 (25 נקודות) :

ממשו את הפונקציה

```
bool is_partition_possible(int arr[], int n)
```

המקבלת מערך של מספרים שלמים ואת גודלו ובודקת האם המערך ניתן לחלוקה לשלישיות של איברים, כך שסכום כל שלישיה בחלוקה הוא אותו מספר.

לדוגמא: עבור המערך הבא

1	11	3	2	4	5	7	3	0
---	----	---	---	---	---	---	---	---

הפונקציה תחזיר true, מאחר וחלוקה הבאה עונה על הדרישה עם סכום כל שלישיה שווה ל-12:

{1, 11, 0}, {3, 4, 5}, {2, 7, 3}

לדוגמא: עבור המערך הבא

1	14	3	2	4	5	7	3	0
---	----	---	---	---	---	---	---	---

הפונקציה תחזיר false, מאחר ולא קיימת חלוקה לשלישיות עם סכום כל אחת שווה ל-13.

הערות:

- יש להשתמש בשיטת backtracking כפי שנלמדה בכיתה.
- בשאלה זו אין דרישות סיבוכיות, אולם כמקובל ב-backtracking יש לוודא שלא מתבצעות קריאות רקורסיביות מיותרות עם פתרונות שאינם חוקיים.
- ניתן ומומלץ להשתמש בפונקציות עזר (ויש לממש את כולן).
- ניתן להניח, כי אורך המערך הינו כפולה של 3.

```
bool is_partition_possible(int arr[], int n)
```

```
{
```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]